

Implementace systému pro zpracování textových zpráv pomocí návrhového vzoru Decorator

Cíl: Implementujte systém pro zpracování textových zpráv pomocí návrhového vzoru Decorator podle přiloženého UML diagramu. Cílem je vytvořit strukturu, která umožní dynamicky přidávat funkce, jako je šifrování, komprese a analýza textu, bez nutnosti měnit základní implementaci zpráv.

Popis zadání: Váš úkol je vytvořit systém pro zpracování textových zpráv, kde můžete:

- Vytvořit základní zprávu s textem.
- Dynamicky přidávat různé funkce (např. šifrování, komprese, počítání slov) pomocí dekorátorů
- Použít vzor Decorator k rozšíření funkcionality zpráv, aniž byste měnili jejich původní implementaci.

Diagram UML, který máte k dispozici, ukazuje strukturu a vztahy mezi třídami. Implementujte třídy a rozhraní podle diagramu.

Požadavky:

1. Vytvořte rozhraní Message:

- Rozhraní bude obsahovat metodu:
 - `String getContent()`: Vrábí obsah zprávy.
- Toto rozhraní budou implementovat jak základní třída pro zprávy, tak všechny dekorátory.

2. Implementujte třídu SimpleMessage:

- Tato třída bude implementovat rozhraní Message a reprezentovat základní zprávu obsahující text.
- Konstruktor třídy přijme obsah zprávy jako parametr.
- Metoda `getContent` vrátí obsah zprávy.

3. Vytvořte abstraktní třídu MessageDecorator:

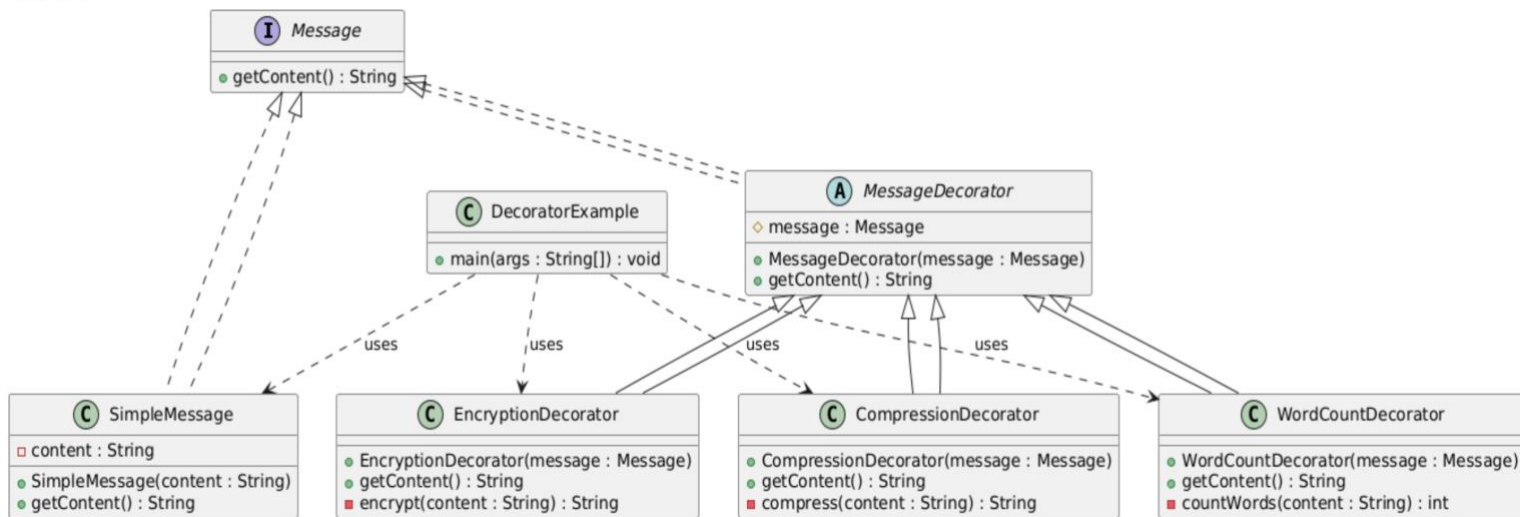
- Tato třída bude implementovat rozhraní Message a sloužit jako základ pro všechny dekorátory.
- MessageDecorator bude obsahovat referenci na Message, kterou dekoruje, a implementuje metodu `getContent` tak, aby vrátila obsah dekorované zprávy.
- Používejte tuto třídu jako základ pro další dekorátory, aby se zajistila jednotná manipulace se všemi typy zpráv.

4. Implementujte konkrétní dekorátory:

- Vytvořte třídy `EncryptionDecorator`, `CompressionDecorator` a `WordCountDecorator` podle UML diagramu:
 - **EncryptionDecorator:** Přidává jednoduché šifrování zprávy. Metoda `getContent` vrátí šifrovaný text. Implementujte jednoduchý šifrovací algoritmus (např. posunutí znaků).
 - **CompressionDecorator:** Přidává kompresi textu. Metoda `getContent` vrátí text bez mezer.
 - **WordCountDecorator:** Přidává informaci o počtu slov ve zprávě. Metoda `getContent` vrátí text s počtem slov na konci (např. "Hello World (Word Count: 2)").

5. Implementujte hlavní třídu DecoratorExample:

- V této třídě vytvořte instanci systému zpracování zpráv podle UML diagramu:
 - Vytvořte základní zprávu typu `SimpleMessage` s textem.
 - Použijte různé dekorátory (`EncryptionDecorator`, `CompressionDecorator`, `WordCountDecorator`) a dynamicky přidávejte funkce ke zprávě.
 - Otestujte různé kombinace dekorátorů a vypište výstupy, abyste demonstrovali, jak se zpráva mění.



```

public class DecoratorExample {
    public static void main(String[] args) {
        // Vytvoření jednoduché zprávy
        Message message = new SimpleMessage( content: "Hello World");
        System.out.println("Original: " + message.getContent());

        // Přidání šifrování
        message = new EncryptionDecorator(message);
        System.out.println("Encrypted: " + message.getContent());

        // Přidání komprese
        message = new CompressionDecorator(message);
        System.out.println("Compressed and Encrypted: " + message.getContent());

        // Přidání počítání slov
        message = new WordCountDecorator(message);
        System.out.println("With Word Count: " + message.getContent());
    }
}

```